JUNE SEMESTER 2026

CSC 2515 / CSC 2517

OBJECT ORIENTED PROGRAMMING

ASSESSMENT

LECTURER: MDM NORADIBAH ADNAN

NAME OF STUDENT	**1- Hussein Musab Abdi** **2- Mohamed Mohamed Hussein**
STUDENT ID	1- 251925406 2- 251925403

Table of Contents

1. Introduction 2

2. Objectives and Scope 2

3. Requirements Analysis 2

4. System Architecture and Class Overview 4

5. UML Class Diagram 5

6. Class, Interface and Abstract-Class Descriptions 7

6.1 Customer, CustomerType and CustomerRegistry 7

6.2 Product and Inventory 7

6.3 CartItem and ShoppingCart 7

6.4 CheckoutRegister 7

6.5 DiscountPolicy, MembershipDiscount and QuantityDiscount 8

6.6 Payment, CashPayment and CardPayment 8

6.7 Receipt and ShoppingSystemApp 8

7. OOP Relationship Justifications 8

7.1 Inheritance 8

7.2 Abstract Class 9

7.3 Interface 10

7.4 Polymorphism 10

7.5 Associations and Multiplicities 11

8. Exception-Handling Design 12

9. Transaction Flowchart 13

10. Explanation of the Transaction Workflow 14

11. Assumptions and Limitations 15

12. Challenges and Lessons Learned 16

13. Conclusion 17

1. Introduction

This report presents Phase 1 (System Analysis and UML Design) of the Shopping System project for CSC 2515 / CSC 2517 Object Oriented Programming. A retail store needs a system that manages customers, inventory items, shopping carts, discounts, payments and receipts. Phase 1 analyses the requirements and designs the system; Phase 2 will implement it in Java.

The brief requires a UML class diagram whose relationships highlight **inheritance**, **abstract classes**, **interfaces**, **polymorphism** and **exception classes**, an explanation of why each class is needed, a justification of the relationships, and a flowchart of a complete transaction. Each concept is used here to solve a specific problem in the shopping system rather than to satisfy a checklist. The terminology and notation follow the subject's lecture slides (Chapters 1–6), for example *fields*, *superclass* and *subclass*, *method overriding* and *dynamic binding*.

2. Objectives and Scope

The objectives of Phase 1 are to identify the main components of the system, propose and justify the classes, design relationships that demonstrate the five required OOP concepts, and produce a UML class diagram and a transaction flowchart.

In scope: a console-based system for a single checkout register that identifies a customer, displays products, manages a shopping cart, applies discounts, accepts cash or card payment, updates stock and produces a receipt. **Out of scope:** the Java implementation (Phase 2), a graphical interface, databases, real card processing, refunds and multiple registers.

Users. The brief does not name roles directly. The **operator** (cashier) enters the customer ID and runs the transaction, and the **customer** selects items and a payment type and is either a regular customer or a member. One operator runs one transaction at a time.

3. Requirements Analysis

Table 1 lists the functional requirements and the design element that satisfies each one. Each requirement is classified as **E** (explicitly required by the brief), **I** (a reasonable inference from the scenario) or **O** (an optional enhancement).

Table 1: Functional requirements and where they are satisfied

ID	Requirement	Type	Satisfied by
FR1	Store item category, description, price and quantity	E	`Product` fields

ID	Requirement	Type	Satisfied by
FR2	Track and update stock levels	E	`Inventory.deductStock()`, `Product.reduceStock()`
FR3	Store customer ID, name and membership status	E	`Customer` fields
FR4	Distinguish regular customers from members	E	`CustomerType` enum, `isMember()`
FR5	Find a customer by ID and display name and status	E	`CustomerRegistry.findCustomer()`
FR6	Display a menu of items with prices	E	`Inventory.displayProducts()`
FR7–9	Add, remove and view items in a cart	E	`ShoppingCart.addItem()`, `removeItem()`, `viewItems()`
FR10	Link one cart to one customer session	E	`ShoppingCart` constructor takes a `Customer`
FR11	Calculate the total cost	E	`calculateSubtotal()`, `calculateTotal()`
FR12	Apply discounts by customer type and number of items	E	`MembershipDiscount`, `QuantityDiscount`
FR13	Change discount rules without changing checkout logic	E	`DiscountPolicy` interface
FR14	Produce the final bill before payment	E	`CheckoutRegister.displayBill()`
FR15–16	Common structure to process and confirm cash and card payments	E	abstract `Payment`, `CashPayment`, `CardPayment`
FR17	Allow future payment methods (e-wallet, online banking)	E	New subclass of `Payment`
FR18	Receipt with customer, items, discount, payment type and amount	E	`Receipt`
FR19	Handle exceptions with user-friendly messages	E	`ShoppingException` hierarchy

ID	Requirement	Type	Satisfied by
FR20	Validate quantity and stock before adding to the cart	I	`ShoppingCart.addItem()`
FR21	Calculate change for cash payments	I	`CashPayment.getChange()`
FR22	Load data and save receipts using text files (Chapter 6)	O	`Receipt.saveToFile()`

4. System Architecture and Class Overview

The design has sixteen classes and six exception classes, arranged in three layers. The **model classes** hold the data and enforce the business rules, such as checking stock. The **controller**, `CheckoutRegister`, coordinates a transaction by calling the model classes in the correct order. The **console interface**, `ShoppingSystemApp`, reads input, displays menus and catches exceptions so that the user sees a clear message instead of a crashed program. Table 2 shows how the classes cover the seven modules in the brief; four supporting classes were added where a clear responsibility required them.

Table 2: Class inventory

Brief module	Class	Type	Responsibility
Inventory for items	`Product`	class	One product and its stock level
	`Inventory`	class	All products; displays them and deducts stock
Customer	`Customer`	class	One customer’s ID, name and type
	`CustomerType`	enum	REGULAR or MEMBER
(supporting)	`CustomerRegistry`	class	All customers; finds one by ID
Shopping Cart	`ShoppingCart`	class	The selected items for one customer
(supporting)	`CartItem`	class	One cart line: a product and a quantity

Brief module	Class	Type	Responsibility
Check-out Register	`CheckoutRegister`	class	Controls totals, discount, bill, payment and receipt
Discount Policy	`DiscountPolicy`	interface	Contract for every discount rule
	`MembershipDiscount`, `QuantityDiscount`	class	The two discount rules
Payment	`Payment`	abstract class	Shared structure of every payment method
	`CashPayment`, `CardPayment`	class	Cash and (simulated) card payments
Receipt	`Receipt`	class	Permanent record of a completed purchase
(supporting)	`ShoppingSystemApp`	class	Console interface with `main`; catches exceptions
(errors)	`ShoppingException` + 5	exception classes	Shopping errors (Section 8)

Every field is `private` and accessed through public methods, following the Chapter 3 advice that all fields should be private. Each class has one responsibility, and `CheckoutRegister` delegates work instead of doing everything itself: the cart totals its own items, each discount rule calculates its own discount, and each payment processes itself.

5. UML Class Diagram

Figure 1 shows the class diagram, drawn in draw.io using the notation from the lecture slides: fields as `- name : type`, methods as `+ method(param : type) : returnType`, and the visibility symbols – (private), + (public) and # (protected). Abstract classes and methods are in *italics*, interfaces are marked «interface», and implementation is a dashed line with a hollow triangle (Chapter 4). Three standard UML symbols not covered in the slides are explained in the diagram key: dashed arrows labelled «create» or «use» are dependencies, underlined members are `static`, and numbers such as `0..*` are multiplicities. Aggregation and composition diamonds are not used because they are not part of the subject material. For readability, dependencies that exist only as method parameters and the `throws` clauses of methods are not drawn; Section 8 lists which methods throw each exception.

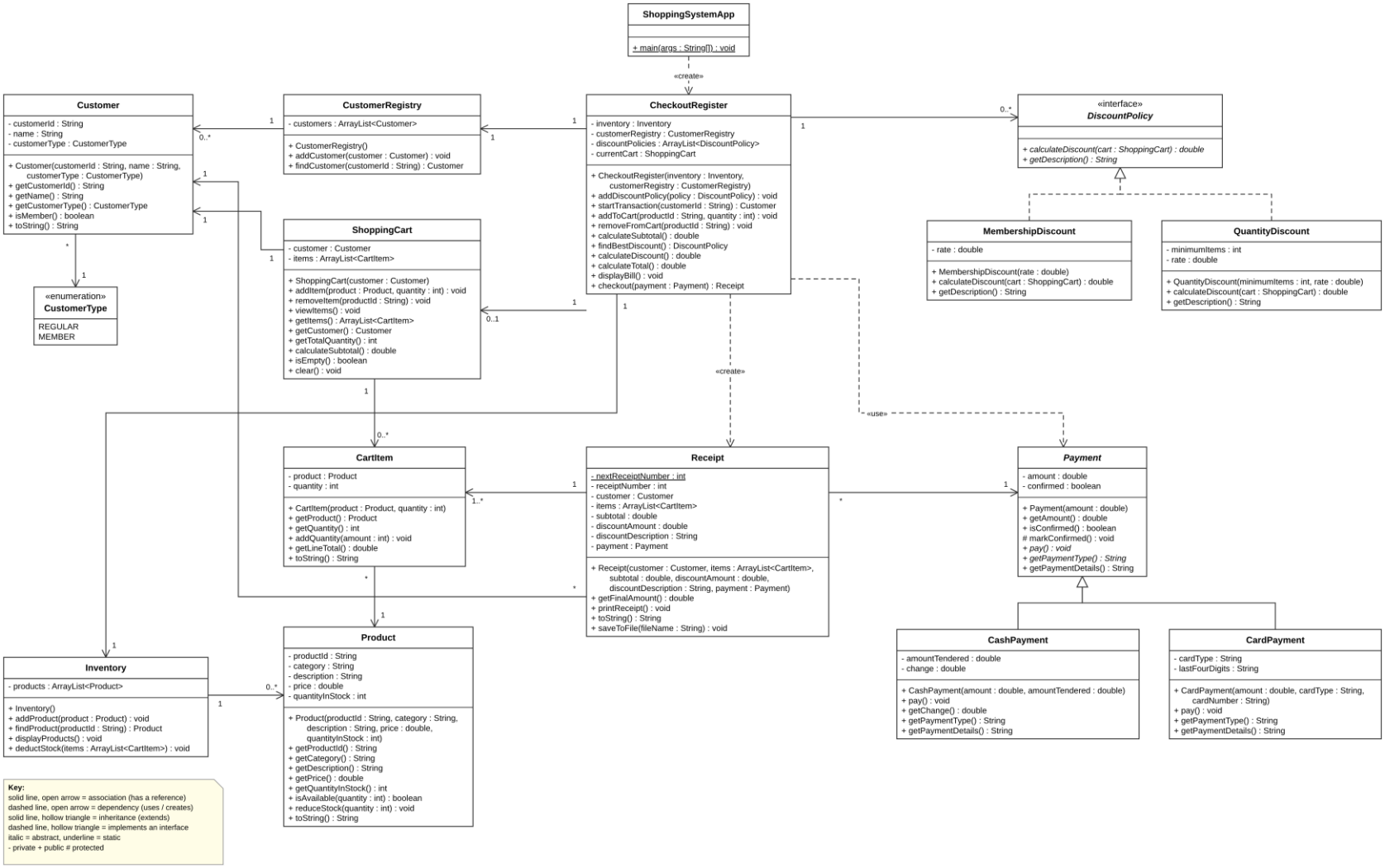


Figure 1: UML class diagram of the Shopping System

6. Class, Interface and Abstract-Class Descriptions

6.1 Customer, CustomerType and CustomerRegistry

`Customer` stores the ID, name and customer type of one customer (FR3). Its method `isMember()` lets the discount rules ask a simple yes/no question without knowing how membership is stored. `CustomerType` is an enumerated type (Chapter 1) with the values REGULAR and MEMBER. An enum was chosen over a `boolean` because the brief speaks of "customer type" and a new type such as VIP could be added later. `CustomerRegistry` keeps all customers in an `ArrayList` (Chapter 2) and finds one by ID (FR5). It is separate from the register so that the register does not take on an unrelated job.

6.2 Product and Inventory

`Product` has the fields named in the brief (category, description, price and quantity in stock) plus a product ID used for selection (FR1). `isAvailable()` checks stock without changing it, and `reduceStock()` lowers it after a purchase, so no other class can set an invalid stock level. `Inventory` is the brief's "Inventory for Items" module: it holds all products, displays the menu with prices (FR6) and deducts stock for a completed purchase (FR2).

6.3 CartItem and ShoppingCart

`CartItem` is one line of the cart: a product and the quantity being bought. This separation is essential. `Product.quantityInStock` is how many units the **shop has**, while `CartItem.quantity` is how many the **customer is buying**; storing both in one field would corrupt the stock figures. `ShoppingCart` holds the cart items for one customer, whose object is required by its constructor (FR10). It adds, removes and views items (FR7–9), checks the quantity and stock before adding (FR20), and provides the subtotal and the total number of units needed by the quantity discount. Only the cart holds the selected items; the register works on the cart rather than keeping its own copy.

6.4 CheckoutRegister

`CheckoutRegister` is the "central controller for transactions" in the brief. It starts a transaction by finding the customer and creating a cart, calculates the subtotal, discount and total (FR11–12), displays the bill before payment (FR14), and in `checkout()` processes the payment, deducts stock, creates the receipt and clears the cart (FR16). It decides the order of these steps but delegates the work to other classes. No separate Bill class is used, because the bill is only a display of three values the register already calculates.

6.5 DiscountPolicy, MembershipDiscount and QuantityDiscount

The interface `DiscountPolicy` declares `calculateDiscount(cart)` and `getDescription()` (FR13). `MembershipDiscount` returns the subtotal multiplied by its rate (for example 10%) when the customer is a member. `QuantityDiscount` returns the subtotal multiplied by its rate (for example 5%) when the cart contains at least a minimum number of units (for example 5). The rates are passed in through the constructor, so the store can change them without editing the classes.

6.6 Payment, CashPayment and CardPayment

The abstract class `Payment` holds the amount and whether the payment is confirmed, and declares the abstract methods `pay()` and `getPaymentType()` (FR15–16). `CashPayment` adds the amount tendered and the change; its `pay()` fails if too little money is given and otherwise calculates the change (FR21). `CardPayment` adds the card type and the last four digits. Card processing is simulated, and only the last four digits are stored, because keeping full card numbers would be poor practice.

6.7 Receipt and ShoppingSystemApp

`Receipt` records a completed purchase: the customer, the items, the subtotal, the discount and the payment (FR18). Its `static` field `nextReceiptNumber` (Chapter 1) gives every receipt a unique number. The receipt keeps **its own copy** of the item list, because the cart is cleared after checkout and a shared reference would leave the receipt empty. `ShoppingSystemApp` contains `main`, reads input with `Scanner`, and catches the exceptions thrown by the other classes to show friendly messages (FR19).

7. OOP Relationship Justifications

7.1 Inheritance

Chapter 3 defines inheritance as an “is a” relationship in which a subclass is a specialised version of its superclass. **A cash payment is a payment, and a card payment is a payment**, so `CashPayment` and `CardPayment` extend `Payment`. They inherit the amount, the confirmed status and the shared methods, add their own fields, and override `pay()`, `getPaymentType()` and `getPaymentDetails()`. Because `Payment` has only a constructor with a parameter, each subclass constructor must call `super(amount)` first. The exception classes form a second inheritance chain: an insufficient-stock error *is a* shopping

error, which *is an* exception. Table 3 shows where inheritance was considered and rejected because the “is a” test fails.

Table 3: Design alternatives that were rejected

Alternative	Reason for rejection	Used instead
`Member extends Customer`	A member behaves no differently; only the discount differs. A customer who becomes a member would also need to become a new object.	`CustomerType` enum
`ShoppingCart extends ArrayList`	A cart *has* items; it *is not* a list. Inheriting would expose list methods that bypass the stock check.	Private `ArrayList` inside the cart
A `Payment` interface as well as the abstract class	It would declare `pay()` twice for no benefit.	Abstract class only
`EWalletPayment` now	The brief describes it as a future method.	Can be added later as a subclass

7.2 Abstract Class

`Payment` is abstract because a payment with no method (neither cash nor card) cannot be processed, so it must never be created directly. Chapter 4 describes an abstract class as “the generic or abstract form” of its subclasses. The subclasses also share real data (the amount and confirmed status) and code, which are written once in `Payment`. An interface cannot hold fields, which is why `Payment` is an abstract class rather than an interface. This also matches the brief, which describes Payment as “a base for subclasses”.

Table 4: Methods of the abstract class Payment

Method	Abstract	Reason
`pay()`	Yes	Every payment must be processed, but there is no general way to do it; each subclass must provide its own version.
`getPaymentType()`	Yes	Only the subclass knows whether it is cash or card.
`getPaymentDetails()`	No	A default (type and amount) exists; subclasses override it and add details by calling `super`.

Method	Abstract	Reason
`getAmount()`, `isConfirmed()`, `markConfirmed()`	No	Identical for all payments. `markConfirmed()` is protected so only subclasses can confirm a payment.

7.3 Interface

`DiscountPolicy` is an interface because Chapter 4 describes an interface as a "contract" that specifies behaviour, and the brief asks for "different discount strategies without changing the checkout logic". The discount rules share no data (a future fixed voucher would not even have a rate), so an abstract class would have nothing to hold. The register depends only on the interface, so a new promotion is one new class that implements `DiscountPolicy`, with no change to the register. A single Discount class full of if-statements would need editing every time a rule changed. Choosing an abstract class for payments but an interface for discounts follows one rule: use an abstract class when subclasses share data and code, and an interface when they share only a contract.

7.4 Polymorphism

Chapter 4 explains that a superclass or interface variable can refer to any subclass object, and that Java uses **dynamic binding** to run the subclass's version of an overridden method at runtime. The design relies on this in four places:

- **Payment.** `checkout()` receives a `Payment` reference and calls `pay()`. The interface creates a `CashPayment` or a `CardPayment`, and the correct `pay()` runs without any if-statement in the register. An e-wallet payment could be added as a new subclass without changing `checkout()` (FR17).
- **Discounts.** The register keeps its rules in a list of `DiscountPolicy` references and calls `calculateDiscount()` on each one; each object applies its own rule. The register keeps the larger discount.
- **Exceptions.** One handler for `ShoppingException` catches all five subclasses, and `getMessage()` returns the specific message of the error that occurred (Chapter 5).
- **toString.** `Product`, `CartItem`, `Customer` and `Receipt` override the `toString()` inherited from Java's `Object` class (Chapter 3) so that they print useful text.

As Chapter 4 notes, a `Payment` variable can only call methods declared in `Payment`; it cannot call `getChange()`, which exists only in `CashPayment`. For this reason, the receipt calls the overridden `getPaymentDetails()`, which for cash already includes the change.

7.5 Associations and Multiplicities

Table 5: Associations, dependencies and multiplicities

Relationship	Multiplicity	Justification
`ShoppingCart` → `Customer`	1 to 1	Each cart belongs to exactly one customer for the session (FR10).
`ShoppingCart` → `CartItem`	1 to 0..*	A new cart is empty; it then holds any number of lines.
`CartItem` → `Product`	* to 1	Each line refers to one product, which stays in the inventory.
`Inventory` → `Product`	1 to 0..*	The inventory holds the store's products.
`CustomerRegistry` → `Customer`	1 to 0..*	The registry holds the store's customers.
`Customer` → `CustomerType`	* to 1	Each customer has exactly one type.
`CheckoutRegister` → `Inventory`, `CustomerRegistry`	1 to 1	Created in `main` and passed in; the register uses but does not own them.
`CheckoutRegister` → `ShoppingCart`	1 to 0..1	No cart before a transaction starts; exactly one during it.
`CheckoutRegister` → `DiscountPolicy`	1 to 0..*	A store may have no promotions or several.
`Receipt` → `Customer`, `Payment`	* to 1	A receipt permanently records who paid and how.
`Receipt` → `CartItem`	1 to 1..*	An empty cart cannot check out, so a receipt has at least one line.
`CheckoutRegister` → `Payment`, `Receipt`	«use», «create»	The payment is only a parameter of `checkout()`; the receipt is created and returned.

8. Exception-Handling Design

Following Chapter 5, the custom exceptions are **checked exceptions**: `ShoppingException` extends `Exception`, the five specific exceptions extend `ShoppingException` (Figure 2), each passes a message to its superclass constructor, and each is declared with `throws` in the methods that can throw it. The model classes throw the exceptions; `ShoppingSystemApp` catches them and displays the message (Table 6).

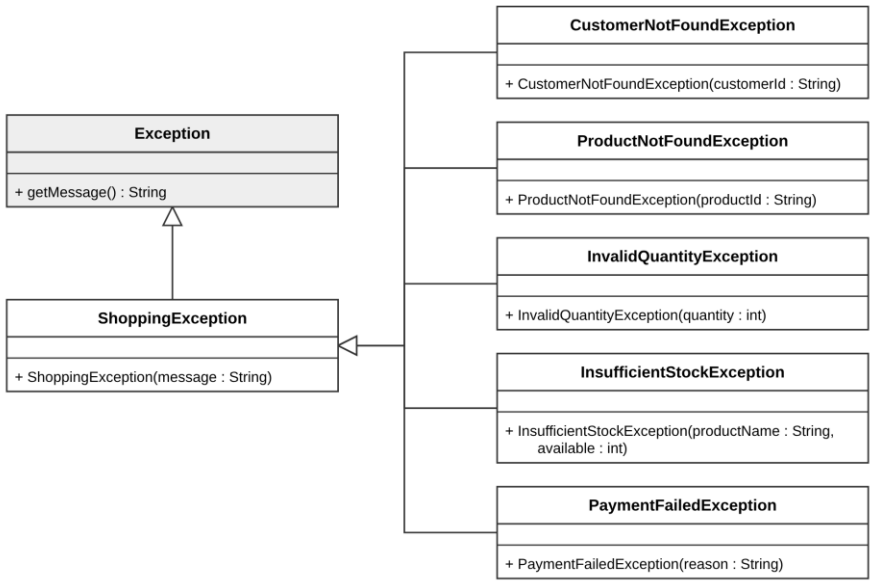


Figure 2: Exception class hierarchy (grey = Java library class)

Table 6: Custom exceptions

Exception	Triggered when	Thrown by	User sees
`CustomerNotFoundException`	No customer has the entered ID	`CustomerRegistry.findCustomer()`	"Customer not found"; enter the ID again or cancel

Exception	Triggered when	Thrown by	User sees
`ProductNotFoundException`	Product code does not exist, or the item is not in the cart	`Inventory.findProduct()`, `ShoppingCart.removeItem()`	"Product does not exist"; enter the product again
`InvalidQuantityException`	Quantity is less than 1	`ShoppingCart.addItem()`	"Quantity must be at least 1"
`InsufficientStockException`	Not enough units in stock	`ShoppingCart.addItem()`, `Product.reduceStock()`	"Only 3 left in stock"
`PaymentFailedException`	Too little cash, or invalid card number	`CashPayment.pay()`, `CardPayment.pay()`	"Payment failed"; try another payment or cancel

Custom exceptions were preferred to returning `null` or `false`, which are easy to ignore and can cause a crash later. A checked exception forces the calling code to deal with the error, and a failed payment guarantees that no stock is deducted and no receipt is created. Ordinary input mistakes are handled by simple validation instead: an invalid menu choice, an empty cart at checkout, and letters typed where a number is expected (Java's own `InputMismatchException`). This keeps the exception design small and easy to explain.

9. Transaction Flowchart

Figure 3 shows a complete shopping transaction as one connected flowchart. The left column identifies the customer and builds the shopping cart; when the customer has finished shopping, a single arrow leads to the right column, which handles checkout and payment. Rounded shapes are start and end, rectangles are processes, parallelograms are input and output, diamonds are decisions with labelled Yes/No branches, and red shapes are error messages. The grey text

under each step names the method in the class diagram that performs it, linking the flowchart to the UML design.

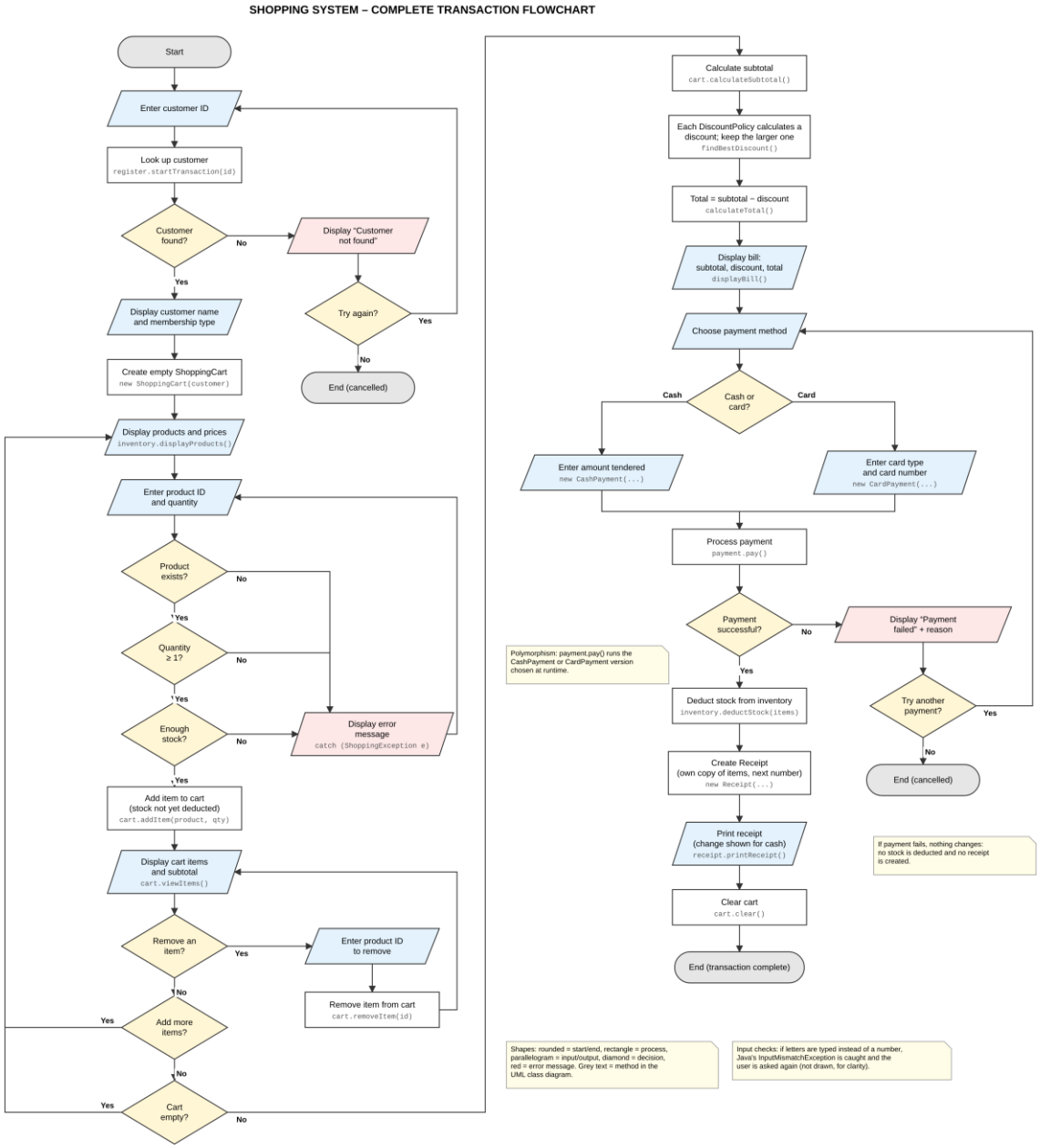


Figure 3: Complete transaction flowchart of the Shopping System

10. Explanation of the Transaction Workflow

Identifying the customer and shopping. The operator enters a customer ID. If it is not found, an error is shown and the operator can try again or cancel. Otherwise the customer's name and membership type are displayed, an empty cart linked to the customer is created, and the products are listed. The operator enters a product ID and quantity, and three checks run in order: the product exists, the quantity is at least 1, and enough stock is available. Every failure

leads to one shared error message, which represents the single `ShoppingException` handler, and the operator enters the product again. A valid item is added to the cart and the cart is displayed. Stock is only *checked* at this point, not deducted. The customer may then remove an item, add more items, or finish shopping; if the cart is empty at that point, the flow returns to the product list.

Checkout and payment. The subtotal is calculated, each discount rule calculates its discount, the larger one is kept, and the bill is shown **before payment**. The customer chooses cash or card, and `pay()` runs the correct version through polymorphism. If payment fails, nothing changes and the customer can try another payment or cancel. If it succeeds, stock is deducted, the receipt is created and printed, the cart is cleared, and the transaction ends.

Worked example. A member buys 6 units with a subtotal of 100.00. The membership discount is 10.00 and the quantity discount is 5.00, so the larger, 10.00, is applied and the total is 90.00. The customer pays 100.00 in cash and receives 10.00 change. The stock is reduced and the receipt shows the items, "Membership discount (10%)", the cash payment with change, and the final amount of 90.00.

Table 7: Coverage of the required OOP concepts

Concept	Where it appears	Evidence
Inheritance	`CashPayment`, `CardPayment` extend `Payment`; exception chain	Figures 1 and 2
Abstract class	`Payment` with abstract `pay()` and `getPaymentType()`	Italic names, Figure 1
Interface	`DiscountPolicy`	«interface» and dashed lines, Figure 1
Polymorphism	`pay()`, the discount rules, exception handling, `toString()`	Figure 1; "Process payment" in Figure 3
Exception classes	`ShoppingException` and five subclasses	Figure 2; error paths in Figure 3

11. Assumptions and Limitations

The brief leaves some business rules open, so the following assumptions were made. None of them affects the class structure:

- The cart is the only owner of the selected items, although the brief describes both the register and the cart as keeping them.
- The **bill** is shown before payment and the **receipt** is created only after payment succeeds, because the receipt must include the payment type.
- Discount rates are illustrative (10% for members, 5% for 5 or more units); "number of items" means total units; only the larger discount applies.
- Stock is checked when an item is added but deducted only after a successful payment.
- Card payment is simulated, and data is held in memory.

Limitations. Refunds, split payments and multiple registers are not supported. Because only one register is modelled, the stock re-check in `deductStock()` cannot fail in practice and is therefore not drawn in the flowchart. E-wallet and online banking are not designed, although the structure allows them to be added as new subclasses of `Payment`.

12. Challenges and Lessons Learned

Inconsistencies in the brief. The brief describes both the register and the cart as holding the chosen items, and asks for "the final bill or receipt before payment" although the receipt must show the payment type. We resolved this by making the cart the single owner of the items and by separating the bill from the receipt. We learned that requirements sometimes need interpretation, and that each interpretation should be stated as an assumption.

Abstract class or interface for Payment. The lecture slides illustrate interfaces with a Payment example, but the brief calls Payment a base for subclasses. Comparing the two options showed that payments share fields, which an interface cannot hold, while discount rules share only behaviour. We learned to choose between an abstract class and an interface by what the classes actually share.

Using inheritance only where it fits. A `Member` subclass looked like an easy way to show inheritance, but the "is a" test showed that a member does nothing differently. We learned that inheritance should model real specialisation, and that an enum is often the simpler choice.

Ownership of data. Putting the purchase quantity on `Product` would have mixed the shop's stock with the customer's order, so we introduced `CartItem`. We also realised that the receipt needs its own copy of the items because the cart is cleared after checkout. Both problems taught us to think carefully about which object owns which data.

Readable diagrams. With 22 classes, the first class diagram was crowded. Moving the exceptions into a separate figure, leaving out parameter-only dependencies and arranging the flowchart in two connected columns made the diagrams clear without losing information.

13. Conclusion

This report has presented the Phase 1 analysis and design of the Shopping System. Every explicit requirement in the brief is traced to a design element. Sixteen classes, each with one clear responsibility, demonstrate all five required OOP concepts in ways that solve real problems in the system. Inheritance and an abstract class structure the payment methods, an interface separates the discount rules from the checkout logic, polymorphism lets the register handle any payment or discount, and checked exceptions report errors clearly. The class diagram and flowchart describe the same system: invalid input never reaches payment, and stock and receipts change only after a successful payment. The design provides a clear basis for the Phase 2 implementation.